Bounding an App's Authority on Soroban: Perch and oz-policy-builder Compared

Willem Wyndham and Elizabeth Engelman

The Aha Company

Abstract. Perch and oz-policy-builder bound what an app may do from an OpenZeppelin-style Soroban smart account in two different ways. oz-policy-builder composes a decision from separate contracts attached to the account's rules: a shared interpreter checks each call against a predicate synthesised from a recorded transaction or a declared intent, OpenZeppelin's `spending_limit` keeps totals, and threshold policies count signers. Perch decides at one point inside the account, with the window cap in the same decision. We compare the two from their public documentation and source at fixed revisions, and from one task run in both in soroban-sim. Both enforce the configured window in that task. Perch integrates the window cap into its decision function and supplies proofs about the function's Lean translation; the account shell remains tested. oz-policy-builder is stronger in maturity, authoring and expressiveness: a testnet deployment, external review, tests on real mainnet transactions, record-and-synthesise authoring, and live reads, clock windows and quantifiers. We end with what Perch should adopt from it.

Keywords: Smart accounts · Authorization · Soroban · Policy composition · Policy synthesis

1 Introduction

An app or agent that acts for a Soroban smart account without a signature prompt for each call needs a bound on what it may do: which contracts and functions, which recipients, how much over time, and until when. Two designs build that bound on OpenZeppelin's account model [4]. oz-policy-builder [7] attaches a shared predicate interpreter to the account's context rules as a policy, beside OpenZeppelin's own policies, and writes its predicates with an off-chain synthesiser. Perch links one evaluator into the account and decides every grant there. The Perch paper, *Perch: Bounded Grants for Apps and Agents on Soroban Smart Accounts*[1], gives the comparison one section; this companion gives it in full, with sources, and ends with what Perch should learn.

Sources and runs. We compare Perch [9] at 0b3752d, whose evaluator is rule-core [8] at a3c92ba; oz-policy-builder at 43b13ed (2026-10-30), release v5.2.0

[1] `https://bitswell-hq.github.io/stellar-registry__perch/perch-paper/`

and 17 later commits, none under its `contracts/` directory; and OpenZeppelin `stellar-contracts` v0.8.0 (ddc327e). Claims about oz-policy-builder are read from its public documentation and source at that pin, cited by path and line (unqualified paths are oz-policy-builder's), or come from our runs. We built it, and its 575 Rust test runs and 1,823 TypeScript tests passed; we ran its record, synthesise, verify and install flow against an OpenZeppelin v0.8.0 account on a local Stellar quickstart network (protocol 30), and the calls outside the rule were refused with the codes its documentation gives; and we ran one task through both designs in soroban-sim [6] (§3).

Disclosure. The authors designed Perch and work at The Aha Company. soroban-sim, the host in which both designs ran, is published at `theahaco/soroban-sim` (Apache-2.0).

Shared ground. Both designs use OpenZeppelin's context-rule model: an authorization entry names, in its `context_rule_ids`, one rule for each context (each call or contract creation the entry authorizes), and that rule alone decides the context. Perch reproduces OpenZeppelin's authorization types and its two signer kinds, *external* (a key a verifier contract checks) and *delegated* (an address whose own authorization the host checks), but attaches no policies. Both judge each context alone, so a call that authorizes two contexts needs a rule for each (`docs/architecture.md:307-309`). Both deny by default.

The trade-off. OpenZeppelin with oz-policy-builder composes a decision from contracts that can be swapped and audited separately; its authors call the interpreter "one audit-once engine" to which "every policy is declarative data" (`README.md:21-24`). Perch puts one grant document behind one decision point: the account's linked evaluator checks target, functions, arguments, threshold, expiry and window cap together, against state held only in the account. Composition buys flexibility and reuse; one decision point gives one place to check and to prove. Eight words mean different things in the two systems (Table 1); we use them in Perch's sense unless a sentence says otherwise.

2 Two Designs

oz-policy-builder. oz-policy-builder has two halves (`README.md:13-24`). Off chain, a synthesiser, `@crediolabs/policy-synth`, records a transaction or takes a declared Intent, writes a predicate in the interpreter's wire form, verifies it through the interpreter's `check` (a dry run that always fails with its verdict), renders a review, and returns an unsigned install transaction; a command-line tool and an MCP server expose the same nine tools. On chain, one shared `policy-interpreter` contract, attached to an OpenZeppelin context rule as a policy, evaluates the stored predicate in `enforce` on every guarded call (`policy-interpreter/src/lib.rs:192-277`). Much of `docs/stellar.md` describes Prime, a custody product built on these parts; the interpreter and the synthesiser correspond to Perch's evaluator and document compiler.

Table 1: Eight words the two systems use differently. Grant, scope, policy, interpreter, `execute`, recovery, cap and expiry mean different things in the two systems. *Scope* is Perch's JSON name for a rule's target. The last column gives the meaning in OpenZeppelin's account or in oz-policy-builder.

Term	In Perch	Name in the artifacts	In OpenZeppelin or oz-policy-builder
Grant	A bounded set of rules the owner installs so an app can act without a prompt for each call	grant	No single object: one context rule per contract, each with its policies
Rule target	The contract whose functions a rule lets be called	JSON `"scope"`	The context rule's `CallContract` address
Policy	—	—	A contract attached to a context rule; the account calls its `enforce` after authenticating (hooks `install`, `uninstall`, `enforce`)
Interpreter	The policy contract of Perch's earlier design (0.5.1); its address is still pinned and reported, never called	`perch_interpreter`	oz-policy-builder's `policy-interpreter`, the shared predicate evaluator
`execute`	The account's owner-authorized call as the account	`execute(target, function, args)`	The adapter's batch call `execute(calls, grants, wait)`
Recovery	Replacing lost credentials through guardians, a zero-knowledge proof, or both	`rcv_*`, `apply_doc`	The Prime app's "recovery" moves custody funds to a fixed address
Cap and window	At most *limit* charged over the last P ledgers: the window at ledger L is $(L - P, L]$	`cap`, `period-ledgers`	`spending_limit` uses the same window boundary. In an oz-policy-builder Intent, `cap` bounds one call (`lte`)
Expiry	`not-after-ledger` N authorizes through N − 1	`not-after-ledger`	OpenZeppelin `valid_until` N authorizes through N

The OpenZeppelin order. An OpenZeppelin v0.8.0 account's `__check_auth` validates each selected rule (it exists, matches the context and has not expired) and refuses any signer on no selected rule; then it authenticates the signers; then it calls `enforce` on each policy attached to the rule, in the rule's stored order, which for policies given at creation is their address order (`storage.rs:629-690` in `stellar-contracts`). The interpreter, `spending_limit` and the threshold policies are such policies, each with its own state keyed by account and rule.

Perch. For ordinary authorization, a Perch account's `__check_auth` calls rule-core's linked `rule_eval::check_auth` on the selected rules and their spending records, which decides the whole authorization, caps included, before any signer is authenticated (Perch `gate.rs:93-113`); it then authenticates external signers through verifier contracts and delegated signers through the host, and writes the new spending records only when authentication also succeeds (`gate.rs:114-121`). An evaluator refusal calls no verifier; a bad signature fails during authentication. Recovery completion uses the controller's separate path. Perch's deployed build

c135aca used the attached-policy shape on a fork of OpenZeppelin's library; the design compared here moves the decision into the account.

Three ideas from oz-policy-builder. A *predicate* is the interpreter's stored expression for one (account, rule) pair; it denies by default and has no `not`, and its `ne` and `nin` deny when an operand does not resolve (`policy-interpreter/README.md:632-639`). An *execution adapter* runs a batch of calls atomically, since Soroban allows one host-function invocation per transaction, and a predicate on its `execute` can relate the batch's calls (`docs/stellar.md:191-195`). A *keyless rule* has no signer; the interpreter admits one only when it is bound to an adapter, and anyone may then run what the predicate permits (`policy-interpreter/src/lib.rs:210-223`). Perch refuses a rule with no signer.

Figure 1 follows one guarded call through both accounts, as read from source, under the game grant of §4: the app may call `guess` on a game, which takes a 1-XLM stake through a transfer it makes, and may pay XLM only to the game, within a cap. Over the cap, Perch refuses in its decision, before any signer is authenticated, and the OpenZeppelin account in `spending_limit`, after authentication. A direct transfer to the game, with no guess, passes in both, because each rule sees its own context alone.

3 One Task in Both Designs

The task is the Perch paper's running example: an app key may send XLM from the account only to the shop, at most 10 XLM in any 100-ledger window. An executable walkthrough[2] that accompanies that paper ran it in one soroban-sim session per design (soroban-env-host 29.0.2 in soroban-sim, protocol 29, applying with enforcing authorization; simulation's recording-mode host build, not the exact-metering one), so each account's `__check_auth` ran on every request. The OpenZeppelin v0.8.0 account's rule carried the predicate oz-policy-builder's synthesiser writes from the task's declared Intent, a per-call bound and the recipient pin, with `spending_limit` beside the interpreter for the window; the Perch account held one grant with a 10-XLM cap per 100 ledgers (about eight minutes). The signers differ by design: oz-policy-builder's rules accept only delegated signers, so the app signs on the OpenZeppelin side as a Stellar account, in a second authorization entry, and on the Perch side with an Ed25519 key that a verifier contract checks.

Both designs gave the same verdict on each of the seven requests (Table 2); they differ in which contracts took part and where each refused (Fig. 2). For a payment, the OpenZeppelin account called `spending_limit` and then the interpreter, and Perch called its verifier. Over the cap, `spending_limit` refused before the interpreter ran, and Perch refused in its decision. To the wrong recipient, `spending_limit` recorded the spend and the interpreter then refused, which rolled the record back with the call. A key signing in the app's name failed the host's signature check in both: on the OpenZeppelin side for the delegated account's

[2] `https://bitswell-hq.github.io/stellar-registry__perch/oz-walkthrough/`

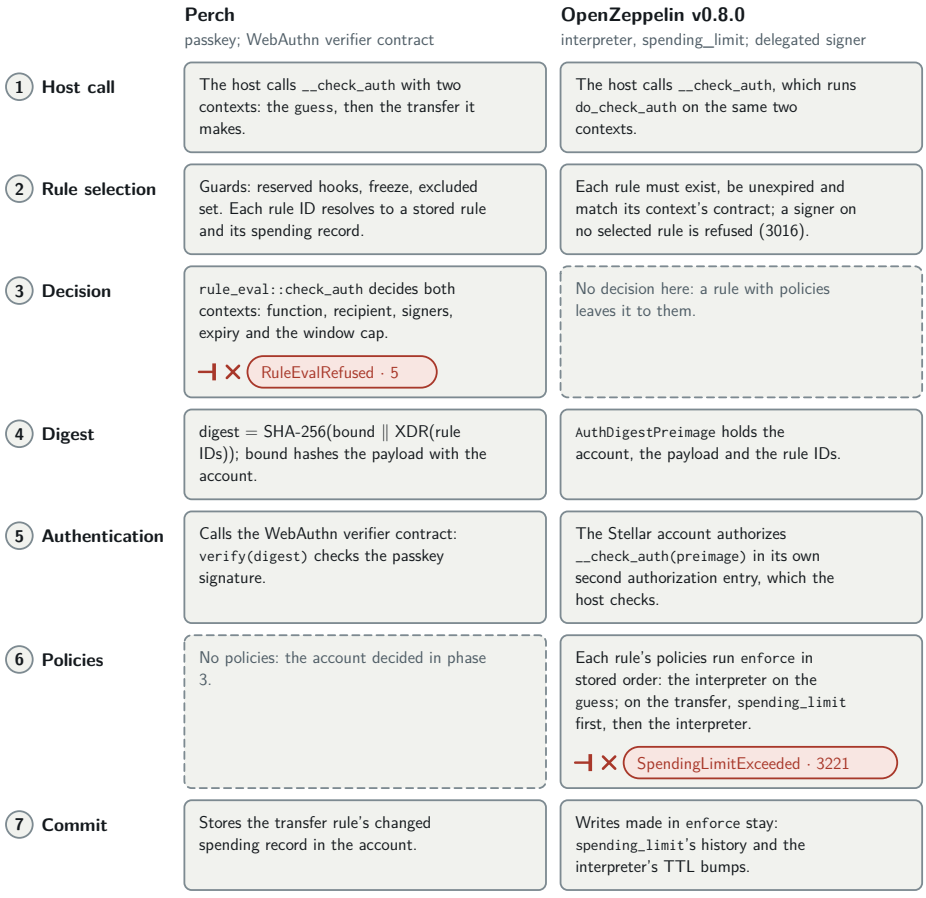


Fig. 1: One guarded call in two accounts. Phase by phase through `__check_auth` for a lost guess, with notes for a guess over the cap and a direct transfer: under Perch, and under an OpenZeppelin account whose rules carry oz-policy-builder's `policy-interpreter` and OpenZeppelin's `spending_limit`. The Perch signer is a passkey checked by a verifier contract. oz-policy-builder's rules accept only delegated signers; here the signer is a Stellar account whose own authorization entry the host checks. The OpenZeppelin account runs a rule's policies in the rule's stored order, which for policies given at creation is their address order. A cross marks where each design refuses a guess over the cap: Perch before any signer is authenticated, the OpenZeppelin account in `spending_limit` after authentication. Dashed: no such phase in that design, or not reached in the scenario. Read from source at Perch 0b3752d, OpenZeppelin `stellar-contracts` v0.8.0 and oz-policy-builder 43b13ed; Table 2 gives the recorded run of the shop task.

Table 2: One task in both designs, run in the walkthrough. The task: an app key may send XLM from the account only to the shop, at most 10 XLM in any 100-ledger window. One soroban-sim session per design (soroban-env-host 29.0.2 in soroban-sim, protocol 29, applying with enforcing authorization; simulation's recording-mode host build, not the exact-metering one), so `__check_auth` ran: the OpenZeppelin v0.8.0 account 8286422f… with oz-policy-builder's interpreter 6704996f… and OpenZeppelin's `spending_limit` 65e70aad…, carrying the rule policy-synth synthesises from the task's declared Intent; and Perch 0b3752d with one grant. Signers: a delegated Stellar account on the OpenZeppelin side, as oz-policy-builder requires; an Ed25519 key checked by a verifier contract on the Perch side. Each refusal is an enforcing simulation that left the ledger unchanged. The steps are those of the walkthrough page: page SHA-256 `a329cc61…`, reference run `62c79b4f…`.

Step	Request	OpenZeppelin account with oz-policy-builder	Perch
1	The owner installs the rule	Applied	Applied
2	The app pays the shop 6 XLM	Applied	Applied
3	5 XLM more in the same window	Refused by `spending_limit`, #3221 `SpendingLimitExceeded`; the interpreter is not called	Refused, auth error 5; no verifier call
4	1 XLM to another recipient	Refused by the interpreter, #100 `ArgMismatch`, after `spending_limit` recorded the spend (rolled back)	Refused, auth error 5; no verifier call
5	Another key signs in the app's name	Refused by the host's signature check on the app's entry; no policy runs	The evaluator accepts; the verifier traps in the host's signature check
6	A key not on the rule signs as itself	Refused by the account, #3016 `UnauthorizedSigner`	Refused, auth error 5; no verifier call
7	100 ledgers later, 5 XLM	Applied	Applied

entry, before any policy ran; on the Perch side when the verifier trapped, after the evaluator accepted.

A per-call bound alone. The interpreter does not "keep rolling totals" (`docs/stellar.md:404-407`). Outside the walkthrough, on a local Stellar quickstart network (protocol 30), an OpenZeppelin account whose rule carried only the interpreter, with a per-call bound of 5 XLM, applied three sends within the bound, moving 9 XLM, and refused the one above it (error 100). This run was not archived; the lower panel of Fig. 2 repeats it.

The recorded dual run. The Perch paper's dual simulation runs the same task with the real contract bytes of both designs side by side, offline, and records a reference run of 15 commands.[3] There the predicate bounds each transfer at 10 XLM. After 6 XLM applies in both and a further 5 XLM is refused in both, the owner removes `spending_limit` from the OpenZeppelin rule; three payments of 6 XLM then each pass the per-call bound, moving 18 XLM within one window,

[3] `https://bitswell-hq.github.io/stellar-registry__perch/perch-paper/sim/`; the Perch paper's section "Both Designs in soroban-sim" describes it.

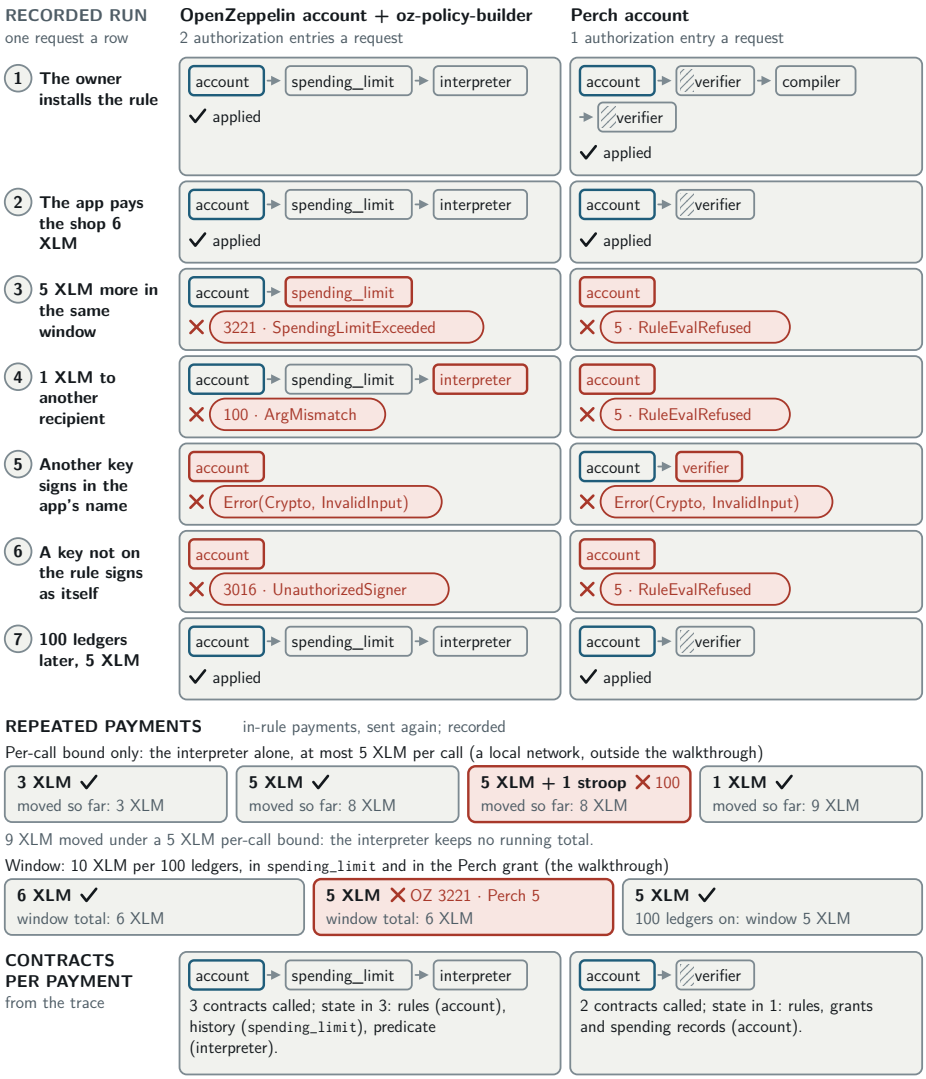


Fig. 2: One task in both designs, replayed from the recorded run. Each row is one request of the walkthrough (Table 2); for each design it lists, in order, the contracts called while the account authorized the request, and where the request was refused. Below the rows, one panel repeats in-rule payments: under a per-call bound alone, an OpenZeppelin account whose rule carries only the interpreter, run on a local network outside the walkthrough, where three sends moved 9 XLM under a 5-XLM bound; and under a window, OpenZeppelin's `spending_limit` or a Perch grant, from the walkthrough. The last panel lists the contracts each design called for one payment and where its decision state lives. The OpenZeppelin side's second authorization entry is its delegated signer's, because oz-policy-builder takes only delegated signers. Calls and codes are recorded, not drawn from source.

while Perch refuses all three because its window already holds 6 XLM. Restoring `spending_limit` starts an empty history, as replacing a capped Perch rule starts an empty window. The difference is a configuration the owner chose: oz-policy-builder documents limits over time as coming from a token allowance and its expiry, or from `spending_limit` beside the interpreter (§5).

4 One Intent, Two Spellings

Figure 3 writes one intent in both systems: the app key may call `guess` on the game, and may send XLM only to the game, at most 2 XLM per 17,280 ledgers (one nominal day), through ledger $N-1$. It is the game grant the testnet preview of the Nido wallet installed through Perch (Perch `release_stack.rs:4019-4030`). oz-policy-builder's side is an Intent (`packages/policy-synth/src/intent.ts:18-59`), the predicate it lowers to for the XLM rule (`intent.ts:117-140`) and the `install_policy` arguments (`src/tools.ts:588-640`), read from source; we did not run these installs. The marks in the figure match the notes.

① Both select one rule per context, and each transfer rule sees only the transfer (`policy-interpreter/README.md:647-649`), so in both a transfer to the game is allowed without a `guess`.
② Perch's window cap is part of the rule; a document cannot bound one call's amount. oz-policy-builder requires a per-call bound (`lte`) unless the owners set `allowUnboundedAmount`, and attaches `spending_limit` for the window, because the interpreter keeps no totals (`dsl.rs:65-72`).
③ Perch installs both rules with one `apply_grant` call; oz-policy-builder builds one `add_context_rule` per `install_policy` call (`tools.ts:588-590`).
④ Perch's rule holds a passkey as an external signer. oz-policy-builder writes every rule signer as delegated (`install.ts:148`) and the interpreter refuses external signers (error 212), so its app signs as a Stellar or contract address; its install is authorized by `sourceAccount`, a signer of rule 0 (`tools.ts:590`). Any account may submit a Perch install that carries the owner's authorization.
⑤ In Perch, `not-after-ledger` N authorizes through N − 1. In OpenZeppelin a rule is refused only when `valid_until` < ledger, so the rule authorizes through `valid_until` (`storage.rs:321-322` in `stellar-contracts`); `validUntilLedger` $N-1$ therefore names the same last ledger.

One more difference has no mark. oz-policy-builder can consult live state, for example a `read` asking a registry whether the recipient is listed (`docs/stellar.md:369-375`); a Perch rule reads nothing outside the account.

5 Safety by Composition

What oz-policy-builder documents. Its interpreter sees one authorized call at a time, so a call that authorizes two contexts needs a rule for each (`docs/architecture.md:307-309`); relating the calls of a transaction is the execution

(a) Perch grant document

```
{ "version": 1,
  "network": "Test SDF Network ; September 2015",
  "signers": [
    { "id": "app", "verifier": "C...WEBAUTHN", ④
      "key": "...65-byte passkey key, hex..." } ],
  "rules": [                                    ③
    { "name": "guess",
      "scope": { "type": "contract",
                 "address": "C...GAME" },
      "principals": { "type": "all",
                      "signers": ["app"] },
      "functions": ["guess"],
      "not-after-ledger": N },                  ⑤
    { "name": "guess-xlm",                      ①
      "scope": { "type": "contract",
                 "address": "C...XLM" },
      "principals": { "type": "all",
                      "signers": ["app"] },
      "functions": ["transfer"],
      "args": [ { "index": 1,
                  "pred": { "type": "address-eq",
                            "address": "C...GAME" } } ],
      "cap": { "limit": "20000000",             ②
               "period-ledgers": 17280 },
      "not-after-ledger": N } ] }               ⑤
```

(b) oz-policy-builder Intent

```
{ "calls": [
  { "contract": "C...GAME", "fn": "guess" },
  { "contract": "C...XLM", "fn": "transfer", ①
    "where": [
      { "constraint": "pin", "at": [1],
        "to": "C...GAME" },
      { "constraint": "cap", "at": [2],       ②
        "max": "20000000" } ] } ] }
```

(c) XLM rule: wire predicate and install arguments

```
["and", [["eq", ["call_contract"], "C...XLM"],
         ["eq", ["call_fn"], "transfer"],
         ["eq", ["call_arg", 1], "C...GAME"],
         ["lte", ["call_arg", 2],           ②
                 {"i128": "20000000"}]]]

{ "network": "testnet",
  "smartAccount": "C...ACCOUNT",
  "sourceAccount": "G...OWNER",
  "intent": { "...": "(b)" },
  "contract": "C...XLM",                     ③
  "signers": ["G...APP"],                    ④
  "validUntilLedger": N - 1,                 ⑤
  "spendingLimit": { "amount": "20000000", ②
                     "periodLedgers": 17280 } }
```

Fig. 3: One intent, two spellings. The intent: the app key may call `guess` on the game, and may send XLM only to the game, at most 2 XLM per 17,280 ledgers, through ledger $N - 1$. (a) Perch: one grant document, installed by one `apply_grant` call. (b) oz-policy-builder: an Intent, lowered to one predicate per contract. (c) The XLM rule's predicate in wire form, and the `install_policy` arguments that attach OpenZeppelin's `spending_limit`; a second call installs the game rule. `C...`, `G...` and N are placeholders; `{ "...": "(b)" }` stands for the Intent in (b), passed as `intent`. Read from source; these two oz-policy-builder installs were not run.

adapter's job, since a rule bound to the adapter is honoured only for a call in a batch the adapter checked (`docs/stellar.md:191-195`). Totals come only from outside the interpreter: it "does not see what a transaction moved, keep rolling totals or count calls", and limits over time come from a token allowance and its expiry or from OpenZeppelin's `spending_limit` beside it (`docs/stellar.md:404-407`), which the installer attaches only to a rule that pins `transfer` on a token (`packages/policy-synth/src/install.ts:957-967`). That policy uses Perch's window boundary (entries at or before C − P are evicted, so the window is (C − P, C]) and keeps at most 815 entries; oz-policy-builder's authors deployed the OpenZeppelin policies they pin and state that they did not audit them (`evidence/README.md:154-157`). OpenZeppelin publishes audit reports for its library; the v0.8.0 diff audit lists `spending_limit.rs` and the smart account's `storage.rs` in scope (`stellar-contracts audits/`, v0.8.0 report, p. 4). Several signers need a threshold policy: a rule with any policy accepts one authenticated signer unless `simple_threshold` or `weighted_threshold` is attached, and the installer requires one unless the owners set `anyOne` (`README.md:122-125`). Changing and ending a rule are separate steps. The interpreter pins a policed rule to its signer set and refuses with error 204 once the set changes, so changing who signs means adding a rule and removing the old one, or removing the policy and adding it again with its admins (`docs/stellar.md:552-554`); rules are removed one at a time; and removing a rule with only rule 0's signatures leaves the interpreter's entry for it, never read again, until it archives, because the interpreter's `uninstall` needs the policy admins' own authorization (`docs/stellar.md:548-562`).

What it buys. The interpreter is fixed code, reviewed once, and a policy is data, so "a new combination of constraints needs no new contract and no re-audit" (`README.md:21-24`). Because each policy is a contract, a predicate can read live state, up to 8 values from other contracts, such as an oracle price or a registry (`dsl.rs:409-423`; `docs/stellar.md:369-375`). OpenZeppelin's own policies attach to the same rule, and one can be added or swapped without changing the others. In exchange, each guarantee holds while the contract that gives it is attached: the window bound while `spending_limit` is on the rule, the threshold while a threshold policy is. Each policy keeps its own state, and a write by an earlier policy is rolled back with the call when a later one refuses (Table 2, step 4).

What Perch does instead, and what that costs. A Perch grant is one document decided by one evaluator linked into the account: target, functions, argument conditions, threshold, expiry, generation and window cap are checked together, against rules and spending records held only in the account. There is nothing to attach or order, and one install writes a grant whose rules span several contracts. The cost is the converse of what composition buys. A rule can say less: five conjunctive argument conditions (`is-self`, `address-eq`, `string-in`, `string-prefix`, `u32-eq`), no disjunction or negation, no bound on one call's amount, and no reads of other contracts. And because the evaluator is linked into the account, a new kind of condition needs a new evaluator, new proofs and a new account build, which an existing account reaches only through its owner's upgrade, at least

120,960 ledgers after it is scheduled (Perch `perch-recovery-interface/src/account.rs:15`).

6 Where oz-policy-builder Is Stronger

oz-policy-builder is ahead of Perch in three respects, which a reader choosing a design today should weigh first.

Maturity. oz-policy-builder pins one interpreter instance on testnet (`packages/policy-synth/src/config.ts:57-80`). Release v5.2.0 is the code submitted for an external audit, which started on 26 October 2026 and is in progress, with the OpenZeppelin account and policies outside its scope (`README.md:57-59`; `docs/audit-overview.md:76-85`). Its off-chain packages were hardened after an external penetration test, in pull request #78 (branch `fix/hacken-pt-findings`), whose merge commit is the pin. The interpreter has 440 tests, 26 of them against OpenZeppelin's v0.8.0 account (`docs/audit-overview.md:15-20`); a corpus test takes 33 real mainnet transaction envelopes, of 26 shapes, through decoding, synthesis and the interpreter (`packages/policy-synth/src/corpus.test.ts:1-3`); scripted testnet runs cover the executor binding, the adapter, a no-gate liquidation and the v0.8.0 migration (`evidence/README.md:10,18-24`); mutation testing of one path gives 31 of 32 killed, the survivor equivalent (`evidence/README.md:23`); and a STRIDE register gives each threat a control and a status. Perch's grant design is merged into its `dev` branch but not yet `main`, runs on testnet only as a preview of an earlier build, and is unaudited.

Authoring. oz-policy-builder starts from what an app did. Its synthesiser records a transaction, or takes a declared Intent, and writes a predicate that permits it; verifies it by dry-running the deployed interpreter's own `check`, so one evaluator both verifies and enforces; drops each conjunct in turn to test that each is needed; renders a review; and returns unsigned install bytes (`packages/policy-synth/README.md`, lines 58 and 142). Its overlap scan reads every live rule of the account and refuses, among other cases, a rule a shared signer could bypass through a looser neighbour, or a total such a neighbour would escape (`policy-synth/README.md:171-176`). The same nine tools run from a command line and from an MCP server, so an agent can drive them, and the packages are published at 5.2.0. Perch starts from what the owner grants: a wallet declares a grant document, which the account's compiler validates and the account lowers against its own authority. Perch has a document builder and offline checks, but no recorder, synthesiser or review renderer.

Expressiveness and ecosystem fit. oz-policy-builder adds rules to accounts already in use, with no new account code: its installer signs for the v0.7.2-line build that existing Prime and mainnet accounts run, and for v0.8.0 (`packages/policy-synth/README.md:143`; `deployments/oz-accounts.json:2`). A Perch grant needs a Perch account. The interpreter's language has `and` and `or`, six comparisons, `in` and `nin`, `all`, `any` and `sum` over vectors, scaled arithmetic, the ledger

time `now`, nested `call_path` and `read` (`policy-interpreter/src/dsl.rs:84-158`). An Intent can state a clock window (`until`, `from`), a least swap output relative to another argument (`minOut`), and conditions on every, some or the total of a vector's elements (`each`, `some`, `total`) (`intent.ts:18-49`). Multi-signer rules use OpenZeppelin's threshold policies. A refusal reports a specific code for each reason (`policy-interpreter/src/storage.rs:27-86`), where Perch reports `StaleGrant` (3) or `RuleEvalRefused` (5). And OpenZeppelin v0.8.0 binds the selected rule IDs for both signer kinds (`storage.rs`, lines 157–175 and 396–412, in `stellar-contracts`), where Perch's delegated signers do not bind them. OpenZeppelin's `spending_limit` can change its limit in place and keep its history (`policies/spending_limit.rs:340-365` in `stellar-contracts`); changing a Perch cap starts an empty record (Perch `docs/DEFERRED.md:298-324`). Shapes Perch lacks: a keyless rule lets anyone run, say, a liquidation the predicate permits (`docs/architecture.md:149`); a rule may follow another account's rule, so an issuer governs a borrower's limits (`docs/audit-overview.md:249-254`); an adapter batch can wait before it runs (`docs/stellar.md:125-140`).

7 Where Perch Is Stronger

Perch offers three architectural advantages, each with a caveat, and one recovery stops every earlier grant.

A limit over time inside the decision. A Perch rule's window cap is charged in the decision that checks the call. The cap theorem (Theorem 6 of *What Perch Proves: An Aeneas-Translated Authorization Evaluator and Its Models*[4]) states that from empty records, in any run, the accepted charges x_k at ledgers t_k to a rule capped at λ per P ledgers stay within the limit in every window:

$$\forall L. \sum_{k:\ \max(0,\, L-P) < t_k \le L} x_k \ \le\ \lambda.$$

In oz-policy-builder the interpreter bounds each call, and the window comes from `spending_limit`, an opt-in policy on `transfer` only; with it removed, the recorded dual run moved 18 XLM in one window while Perch held to its cap (§3). Both enforce the window when configured with one: the difference here is where the cap is expressed and checked, not whether a window can be enforced. *Caveat:* the theorem is about the evaluator's Lean translation; that the shell selects rules and persists records as it assumes is argued and tested, not proved (obligations O2 and O6). On chain a record holds at most 907 spends per window, and replacing a capped rule starts an empty window.

One decision point. All the state that decides a grant's call lives in the account: rules, grants, spending records and the grant generation (Perch `lib.rs:597-636`). The decision comes before authentication, so a call refused by the evaluator

[4] `https://bitswell-hq.github.io/stellar-registry__perch/perch-paper/verification/`

invokes no verifier and writes nothing. A bad signature instead fails during authentication. One `apply_grant` installs a grant whose rules span several contracts, and a grant rule may name a passkey or Ed25519 key as an external signer, which the interpreter refuses in the rules it polices. *Caveat:* one point decides only what the evaluator can express (§5).

A proved core. The evaluator is about 2,500 lines of `no_std` Rust in a subset that Charon [1] and Aeneas [2] translate into Lean. Kernel-checked theorems state that the translation returns what a readable model returns on every input, without panicking or diverging (Theorem 1); that when the decision function accepts, each call is allowed by some rule it was given (Theorem 2); that attenuation never widens the calls a grant allows (Theorem 4); the window bound above (Theorem 6, for fixed rules and empty initial records); and that no grant produced by scoped lowering, nor one derived from it, allows a call to the account or the recovery contracts excluded at install (Theorem 8). The shell rechecks the current excluded set when recovery configuration changes. We found no formal model in oz-policy-builder's tree (a search, not a proof of absence); its evidence is testing, partly on the shipped Wasm. *Caveat:* the theorems are about the translation, not the compiled Wasm; the shell is tested, not proved; attenuation cannot yet be installed; and nothing in Perch has been audited.

Stopping every earlier grant. A completed recovery advances the grant generation, after which no earlier grant, nor one derived from it, accepts a nonempty authorization (Theorem 10). *Caveat:* this is a theorem about an account model with no Rust counterpart; the shell's part is tested (O3). oz-policy-builder has no account-key recovery, a difference in scope, not a defect.

8 Lessons for Perch

We propose six changes; none is built.

1. **Recording as authoring.** A front end that, like oz-policy-builder's synthesiser, starts from a recorded transaction and emits a Perch grant document for the owner to review. A recording shows calls, not intent, so the owner would still choose the cap, its window and the expiry.
2. **Simulating before install.** A wallet that installs a grant only after running the intended calls through the deployed account's own evaluator in enforcing simulation, as oz-policy-builder verifies a predicate with the deployed interpreter's `check`.
3. **Minimality checks.** A check that flags a rule wider than the recorded calls need, as oz-policy-builder drops each conjunct in turn, and an overlap scan that refuses a capped rule a key it names could bypass through another rule, since a Perch key holds the union of every rule naming it.
4. **Keeping a record across a limit change.** Keeping a rule's spending record when only its limit changes, as OpenZeppelin's `spending_limit` does.

5. **A corpus of real transactions.** Real Soroban transactions run through Perch's compiler, lowering and evaluator, beside the 58 Lean-generated traces its integration suite runs today.
6. **An audit before claiming maturity.** An external audit of the Perch contracts (account, compiler, recovery controller, verifiers, zero-knowledge pool and adapter, registry) and of rule-core's evaluator. Until then the authors recommend that Perch not hold real funds.

9 Dimensions

Table 3 (Appendix A) sets the two designs side by side on twenty dimensions, each cell with its source and a mark; nothing in it was measured. Its *Assurance* row compares proofs about a translation of Perch's decision function with tests of oz-policy-builder's shipped contracts; neither ranks above the other, and neither is an audit.

10 What This Comparison Does Not Show

What was run. Both runs of the task (§3) are one task, on soroban-sim's recording-mode host build without exact metering, with different signer kinds on the two sides; they show verdicts and call structure, not cost.

Proposed measurement. We propose, and have not run, five workloads under one harness with exact metering, one host version and Ed25519 signers on both sides, reporting CPU instructions, memory, ledger I/O, fees and cross-contract calls: W1, one capped transfer; W2, a losing guess and the transfer it makes; W3, one grant install; W4, ending authority over k grants (one recovery against k calls to `remove_context_rule`); W5, storage after n transfers.

What we do not claim. The recorded call traces describe this fixture only. Until W1–W5 run, we claim no general reduction in calls, cost, speed or code size. We do not claim that Perch binds more in general, or that oz-policy-builder is unproven or runs on mainnet (OpenZeppelin's v0.7.2 policies do). We do not claim that `spending_limit` and Perch's cap behave alike beyond their shared eviction boundary, or that Perch lacking the features behind Table 4's oz-policy-builder rows is a security property.

11 Related Work

Composition is common on EVM chains: ERC-7579 [11] accounts install validators and hooks as modules, Smart Sessions (Rhinestone and Biconomy) restricts a session key through policies [5], and MetaMask's delegation framework restricts a delegation through caveat enforcers [3]. Synthesis from recordings relates to policy mining from logs [10].

12 Conclusion

In the recorded dual run the designs differed only where the owner removed `spending_limit`. oz-policy-builder leads in maturity, authoring and expressiveness; Perch integrates the window cap into a decision function whose Lean translation is proved, with the account shell tested separately. The lessons (§8) are proposed work.

A Sources and Open Items

Table 3 gives all twenty dimensions with their sources and marks; Table 4 lists the open items each project documents about itself, Perch's first.

Comparator artifacts. oz-policy-builder pins one interpreter, on testnet only (mainnet: none pinned), at `CDER6P73JWMIVD3X6TGGOQIDAH3DLU7OKRTLT4ULPTZ3QJJ3QUGTK7CM`, running Wasm `6704996f626904a4e33b95bda17b73963fc9cf502ffdfdbe9d85b4b10288e0d4` with deploy-time limits reads 8, leaves 400, bytes 24,576, elems 32, in 32, depth 5; its execution adapter runs bf85432c…. It signs for two OpenZeppelin account builds, the v0.8.0 build 8286422f… and the v0.7.2-line build 91a2cd56…. The OpenZeppelin policies it pins are v0.8.0, deployed 26 October 2026, on testnet, and v0.7.2, deployed 9 June 2026, on mainnet. `spending_limit` keeps at most 815 history entries at v0.8.0 and 1,000 at v0.7.2, and refuses with 3221. The interpreter's refusal codes include 100 (an argument mismatch), 202 (an install nonce that is not the next), 204 (a changed signer set), 208 (a predicate-hash mismatch), 210 (no authenticated signer), 212 (an external signer) and 218 (a rule that reaches the account's administration). An install's authorization window is 300 ledgers by default, at most 17,280 ledgers (`README.md:42-62`; `config.ts:26-28,57-80`; `deployments/oz-policies.json:2-28`; `policy-interpreter/src/storage.rs:46-111`). Its threat register lists E-25, T-05, T-19, E-36 as open at High (`evidence/stride-threat-model.md:392-405`).

Table 3: Perch and oz-policy-builder, all twenty dimensions with sources. Perch paths are at 0b3752d, oz-policy-builder paths at 43b13ed, OpenZeppelin (OZ) paths at `stellar-contracts` v0.8.0 unless marked. Mark: V checked in source at the pin by the authors; R cited from source by an internal reader and not re-read by the authors; U not verified. Nothing here was measured; the policy order was also observed in the walkthrough. T*n* and O*n* are the theorems and obligations of What Perch proves.

Dimension	Perch	oz-policy-builder on an OpenZeppelin account	Mark
Where the Linked `rule_eval::check_auth` inside `__` decision runs	`check_auth` (`gate.rs:93-105`)	The account selects the rule and authenticates, then calls each attached policy's `enforce`; the shared interpreter evaluates the stored predicate (OZ `storage.rs:629-690`; `policy-interpreter/src/lib.rs:192-277`)	V

Dimension	Perch	oz-policy-builder on an OpenZeppelin account	Mark
Order of checks	Whole rule decision, caps included, before authentication; a batch refused by the evaluator calls no verifier (gate.rs:93-113)	Rule existence, scope and expiry, then a refusal of any signer on no selected rule, before authentication; then each attached policy's enforce (predicate, spending total, threshold) in the rule's stored order: address order for the policies it was created with, then any added later (OZ storage.rs:629-689, 830, 1304, 1588-1596; observed in the walkthrough)	V
Where decision state lives	Account storage only: rules, grants, spending records, generation (lib.rs:597-636)	Account (rules, signers); interpreter, one entry per (account, rule) (policy-interpreter/src/storage.rs:167-188); spending_limit, history per (account, rule) (OZ spending_limit.rs:143-148)	V/R
Conditions on one call	Target, function list, five conjunctive argument conditions, threshold, expiry; no per-call amount bound (perch-ir/src/doc.rs:281-304; rule-core lowering.rs:17-20)	And/or, six comparisons, in/nin, all/any/sum over vectors, scaled arithmetic, ledger time now, nested call_path, read (policy-interpreter/src/dsl.rs:84-158); per-call lte on amounts	V
Relating calls in one transaction	A rule may require its entry's root call to be one contract and function (exact roots, Perch #69 and rule-core #17); otherwise each context is checked alone against its rule	One predicate over the execution adapter's batch argument can relate its calls (docs/stellar.md:350-352; policy-interpreter/README.md:390-397)	V
External state in the decision	None; reads account storage only	read calls other contracts during enforce, up to the instance's max_reads (8 on the testnet pin) (dsl.rs:409-423; config.ts:66-73); same-transaction manipulation is open at High, with the safe-list refusal in place (T-05, stride-threat-model.md:397)	V
Rolling spending cap	In the evaluator; window (L − P, L]; proved from empty records (T6); at most 907 spends per window on chain (docs/recovery/budgets.md:604-611)	Not in the interpreter (dsl.rs:65-72). OpenZeppelin spending_limit beside it: same window boundary, at most 815 entries, transfer only (OZ spending_limit.rs:9-12, 169; install.ts:957-967); the oz-policy-builder authors did not audit it (evidence/README.md:157); OpenZeppelin's v0.8.0 diff audit lists it in scope (OZ audits/, v0.8.0 report, p. 4)	V
Threshold and signer kinds	Threshold inside the rule, counting distinct identities (T3); External (Ed25519, WebAuthn) and Delegated (CAP-0071) signers	A rule with any policy accepts one signer unless OpenZeppelin's threshold policy is attached; the builder refuses that shape unless anyOne (tools.ts:590). Policed rules accept Delegated signers only (lib.rs:455-462, error 212)	V
What a signature binds	External: account and rule IDs. Delegated: account, not rule IDs (gate.rs:106-113; auth.rs:47-61)	v0.8.0: account and rule IDs for both kinds; an external signer signs the hash of AuthDigestPreimage, a delegated signer authorizes it through require_auth_for_args (OZ storage.rs:150-175, 396-412). 0.7.2-line: rule IDs, not the account, for External with a legacy Address credential (OZ v0.7.2 storage.rs:492-495)	V

Dimension	Perch	oz-policy-builder on an OpenZeppelin account	Mark
Ending authority in bulk	A completed recovery advances the grant generation, and every earlier grant refuses in that transaction (T10); otherwise one grant at a time	Rules are removed one at a time. A policed rule is pinned to its exact signer set (error 204 after any change). Removing a rule with only rule 0's signatures leaves the interpreter's entry for it until the entry archives (docs/stellar.md:548-562); a rule that follows it keeps enforcing it (policy-synth README.md:144)	V
Assurance	Lean proofs of the evaluator over its translation (What Perch proves); shell tested; no external audit completed	440 interpreter tests (26 against OpenZeppelin's 0.8.0 Wasm), parity corpus, worst-case suite on Wasm, STRIDE model, mutation testing of one path; no formal model found in the tree (a search, not a proof of absence); pull request #78 hardened the off-chain packages after an external penetration test; external audit of v5.2.0 in progress since 26 October 2026, with the OpenZeppelin account and policies out of scope, though OpenZeppelin's own v0.8.0 diff audit lists spending_limit.rs and the account's storage.rs in scope (README.md:57-59, 166-171; audit-overview.md:15-20, 76-85; evidence/README.md:23; OZ audits/, v0.8.0 report, p. 4)	V
Deployment	This design: testnet preview. Earlier design c135aca: testnet and mainnet	Interpreter pinned on testnet only; no mainnet pin (config.ts:57-80). OpenZeppelin v0.7.2 policies on mainnet (evidence/README.md:154-157)	V
Rules with no signer	Refused, UnguardedRule (rule_set.rs:53-55)	Allowed when bound to an execution adapter; anyone may then run what the predicate permits (lib.rs:210-223; architecture.md:149)	V/R
Rules shared across accounts	None; grant rules are copied into the account at install	A rule may follow another account's rule, whose admins can change the predicate (E-38, Medium, accepted) (lib.rs:76-84; audit-overview.md:243-278)	V
Account-key recovery	Guardian or ZK recovery replaces credentials (The Perch vision)	None; Prime's "recovery" moves custody funds to a fixed address (architecture.md:81-94)	V
Install ordering	apply_doc takes an optional expected revision; apply_grant takes none (lib.rs:1000-1006; docs/account-interface.md:96-135)	install_nonce must be the stored nonce plus one (lib.rs:144-145)	V
Authoring tools	perch-js builder (packages/perch-js/README.md:31-44); on-chain compiler; offline z3 checks in perch-analyze (crates/perch-analyze/src/lib.rs:1-27)	Recorder, synthesiser, verifier with minimality, review renderer, unsigned install builder, overlap scan; nine tools over a CLI and an MCP server (README.md:13-24)	V/R
Who can change deployed code	Account: the owner, after 120,960 ledgers (perch-recovery-interface/src/account.rs:15). Compiler, verifiers, controller: no admin (docs/recovery/spec.md:100). Registry: admin-upgradeable (perch-constructorless-registry/src/lib.rs:105-108)	Interpreter, adapter, gate and seat: no admin and no upgrade path (architecture.md:153). Oracles on the safe list are admin-upgradeable (config.ts:215). OpenZeppelin account and policies: not examined	V/R/U
Keeping state alive	Rule-set TTL extended on the ordinary evaluator path; grants on read; spending records to the maximum on write; renew omits grants (gate.rs:64, 114-119; lib.rs:936-959)	Interpreter entry bumped on a permit when fewer than 100 ledgers remain; instance and read targets need a keep-alive job (lib.rs:256-265; docs/stellar.md:304-308)	V/R

Table 4: Disclosed open items in both systems. Items each project documents about itself. oz-policy-builder rates its items in a STRIDE register; Perch keeps none, so its rows give the status its documents state. H-1/E-36 and E-38 arise from features Perch lacks; their absence in Perch is not a security property of Perch.

System	Item	Status	Source
Perch	The completion marker is consumed by the target's canonical identity, not its exact bytes	open	issue #15
Perch	Delegated signers do not bind rule IDs	by design (CAP-0071)	`docs/recovery/cap-0071.md:28`
Perch	Replacing a capped rule, or changing its cap, starts an empty window	documented	`docs/DEFERRED.md:298-324`
Perch	A rule edited in place between signing and inclusion decides under its new contents	documented	`docs/DEFERRED.md:227-262`
Perch	Grant installs take no expected version	stated	`lib.rs:1000-1006`
Perch	One key is admin and manager of `perch/verifiers` on testnet and mainnet; the admin can replace the registry's code at once	deployment fact	`deployments/*.json:22-23`
oz-policy-builder	H-1/E-36: a keyed door on a no-gate adapter without the grants pin	High, residual Medium; tooling-only guard	`audit-overview.md:165-177`
oz-policy-builder	E-38: a keyless follower's fence can be loosened by the followed rule's admins	Medium, accepted	`stride-threat-model.md:400`
oz-policy-builder	T-05: same-transaction read manipulation	High where a rule uses `read`	`stride-threat-model.md:397`
oz-policy-builder	T-19: the review reaches the owner through the agent	High until the owner side runs it	`stride-threat-model.md:398`

Dimension	Perch	oz-policy-builder on an OpenZeppelin account	Mark
What a refusal reports	`StaleGrant` (3) or `RuleEvalRefused` (5); a wallet re-runs the evaluator off chain for the reason	A specific deny code per reason (`storage.rs:27-86`)	V

Open items. Each project documents items about itself. oz-policy-builder rates its items in its STRIDE register; Perch keeps no register, so its rows give the status its own documents state.

References

1. Ho, S., Boisseau, G., Franceschino, L., Prak, Y., Fromherz, A., Protzenko, J.: Charon: An analysis framework for Rust. arXiv:2410.18042 (2024), https://arxiv.org/abs/2410.18042
2. Ho, S., Protzenko, J.: Aeneas: Rust verification by functional translation. In: Proceedings of the ACM on Programming Languages (ICFP) (2022), https://arxiv.org/abs/2206.07185
3. MetaMask: Delegation Framework, revision b4b8c41. GitHub, MetaMask/delegation-framework (2026), https://github.com/MetaMask/dele

gation-framework/tree/b4b8c41fc912455a1af2300bfcd1276676b92a2e, delegations restricted by caveat enforcer contracts
4. OpenZeppelin: Stellar smart account contracts, v0.8.0 (ddc327e). GitHub, OpenZeppelin/stellar-contracts, packages/accounts (2026), https://github.com/OpenZeppelin/stellar-contracts/tree/ddc327eb92d63709436da3a8455f641c2d5476c5/packages/accounts/src, context rules, signers and policies
5. Rhinestone, Biconomy: SmartSession: a session-key module for ERC-7579 accounts, revision f5aaf86. GitHub, erc7579/smartsessions (2025), https://github.com/erc7579/smartsessions/tree/f5aaf867f7e22f3b9d746ce6f404f3a56833757f
6. soroban-sim, revision d029384. GitHub, theahaco/soroban-sim (2026), https://github.com/theahaco/soroban-sim/tree/d02938430f0ac41f34e5e3e4c8c48d85b871bd68, portable host and session driver; host 29.0.2
7. Untangled Finance: OZ Policy Builder, revision 43b13ed (v5.2.0 line). GitHub, untangledfinance/oz-policy-builder (2026), https://github.com/untangledfinance/oz-policy-builder/tree/43b13ed1d9c6662afd74f446006e69bfcf21259b, policy synthesiser, policy-interpreter contract, execution adapter, custody gate and Prime seat
8. Wyndham, W.: soroban-rule-core, revision a3c92ba. GitHub, bitswell/soroban-rule-core (2026), https://github.com/bitswell/soroban-rule-core/tree/a3c92ba074dcac4052727b7599e5fd8674af6c0a, Rust evaluator, model, translation and proof trust inventory
9. Wyndham, W., Engelman, E.: Perch source, revision 0b3752d. GitHub, bitswell-hq/stellar-registry___perch (2026), https://github.com/bitswell-hq/stellar-registry__perch/tree/0b3752d8a179a5952b05fbebd29131dbbf99fa80, account, compiler, recovery, verifiers and canonical-document proof
10. Xu, Z., Stoller, S.D.: Mining attribute-based access control policies. IEEE Transactions on Dependable and Secure Computing **12**(5) (2015), https://doi.org/10.1109/TDSC.2014.2369048
11. zeroknots, Kopp, K., Lee, T., Makarov, F., Poon, E., Min, L.: ERC-7579: Minimal modular smart accounts. Ethereum Improvement Proposals (2023), https://eips.ethereum.org/EIPS/eip-7579, draft